

# COMPUTER PROGRAMMING IN C++

Official Assignment Solutions: Question 2 through End

## QUESTION 2: Refactored & Modular ATM Simulation System

As required, the monolithic implementation from class has been separated into clean, modular components using explicit function parameters, descriptive `camelCase` variable naming notations, accurate status mapping formatting, and rigorous input array overflow validations.

```
/*
 * File: atm_simulation.cpp
 * Description: Clean, function-based modular refactoring of the class ATM simulation.
 */
#include <iostream>
#include <string>

using namespace std;

// Global State Configurations
const int MAX_TRANSACTIONS = 100;
string transactionHistory[MAX_TRANSACTIONS];
int transactionCount = 0;
int accountBalance = 0;

// Function Layout Prototypes
bool verifyPin();
void displayMainMenu();
void checkBalance();
void depositMoney();
void withdrawMoney();
void viewTransactionHistory();
void logTransaction(string details);

int main() {
    // Title UI initialization utilizing standard terminal colors
    cout << " [36m";
    cout << "                      =====>" << endl;
    cout << "                      BK ATM SYSTEM          " << endl;
    cout << "                      =====>" << endl;
    cout << " [0m" << endl;

    // Authentication check execution block
    if (!verifyPin()) {
        return 0;
    }

    // Execution routing dashboard environment
    int selection;
    do {
        displayMainMenu();
        cout << "
Enter your choice: ";
        cin >> selection;

        switch (selection) {
            case 1: checkBalance(); break;
```

```

        case 2: depositMoney(); break;
        case 3: withdrawMoney(); break;
        case 4: viewTransactionHistory(); break;
        case 5: cout << " [33m
Thank You For Using BK ATM [0m
" << endl; break;
        default: cout << " [31m
Invalid Choice. Try again. [0m
" << endl;
    }
} while (selection != 5);

return 0;
}

```

```

bool verifyPin() {
    const int defaultPin = 12345;
    int enteredPin;
    int maxAttempts = 3;

    do {
        cout << "          Enter your PIN: ";
        cin >> enteredPin;
        maxAttempts--;

        if (enteredPin == defaultPin) {
            cout << " [32m
Login Successful [0m" << endl;
            cout << " [35m Welcome to BK ATM [0m
" << endl;
            return true;
        } else {
            cout << " [31m
Wrong PIN [0m" << endl;
            if (maxAttempts > 0) {
                cout << " [33mRemaining Attempts : " << maxAttempts << " [0m
" << endl;
            }
        }
    } while (maxAttempts > 0);

    cout << " [31m
ATM BLOCKED [0m
" << endl;
    return false;
}

void displayMainMenu() {
    cout << "
[34m===== MAIN MENU ===== [0m" << endl;
    cout << " [1] Check Balance" << endl;
    cout << " [2] Deposit Money" << endl;
    cout << " [3] Withdraw Money" << endl;
    cout << " [4] Transaction History" << endl;
    cout << " [5] Exit" << endl;
    cout << " [34m===== [0m" << endl;
}

void checkBalance() {
    cout << " [32m
Current Balance: " << accountBalance << " FRW [0m
" << endl;
}

void depositMoney() {
    int amount;
    cout << "
Enter amount to deposit: ";
    cin >> amount;

    if (amount <= 0) {
        cout << " [31m
Invalid deposit value. [0m
" << endl;
        return;
    }
}

```

```

    accountBalance += amount;
    cout << " [32m
Deposit Successful" << endl;
    cout << "New Balance: " << accountBalance << " FRW [0m
" << endl;
    logTransaction("Deposited: " + to_string(amount) + " FRW");
}

void withdrawMoney() {
    int amount;
    cout << "
Enter amount to withdraw: ";
    cin >> amount;

    if (amount <= 0) {
        cout << " [31m
Invalid transaction limits. [0m
" << endl;
        return;
    }

    if (amount > accountBalance) {
        cout << " [31m
Insufficient Funds [0m
" << endl;
    } else {
        accountBalance -= amount;
        cout << " [32m
Withdrawal Successful" << endl;
        cout << "Withdrawn Amount: " << amount << " FRW" << endl;
        cout << "Remaining Balance: " << accountBalance << " FRW [0m
" << endl;
        logTransaction("Withdrawn : " + to_string(amount) + " FRW");
    }
}

void viewTransactionHistory() {
    cout << "
[36m===== TRANSACTION HISTORY ===== [0m" << endl;
    if (transactionCount == 0) {
        cout << " [31mNo Transactions Found [0m
" << endl;
    } else {
        for (int i = 0; i < transactionCount; i++) {
            cout << (i + 1) << " " << transactionHistory[i] << endl;
        }
        cout << endl;
    }
}

void logTransaction(string details) {
    if (transactionCount < MAX_TRANSACTIONS) {
        transactionHistory[transactionCount] = details;
        transactionCount++;
    } else {
        for (int i = 1; i < MAX_TRANSACTIONS; i++) {
            transactionHistory[i - 1] = transactionHistory[i];
        }
        transactionHistory[MAX_TRANSACTIONS - 1] = details;
    }
}

```

## QUESTION 3: Menu-Driven "Foundations and History of C++" System

This implementation provides a book system that uses `std::ifstream` input streams to dynamically load text components from external text modules while ensuring execution continuity through built-in standard fallback mechanisms if file assets are missing from the system path.

### Required External Text Components Setup

For file stream operations to function correctly, create the following text modules in your runtime project workspace directory:

- `history_origin.txt` : Enter descriptive details regarding early C extensions, object methodologies, and the milestones leading up to modern standard templates.
- `concept_datatypes.txt` : Detail variable definitions, memory limits for primitives, type modifiers, and pointer layout initializations.

```
/*
 * File: book_system.cpp
 * Description: Interactive structural book manual using standard file streams.
 */
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// Operational Interfaces Prototypes
void displayBookHeader();
void openHistoryModule();
void openConceptsModule();
void streamFileContents(string fileTarget, string fallbackText);
void drawTimelineTree();

int main() {
    int inputSelection;
    do {
        displayBookHeader();
        cout << " [1] History of C++
";
        cout << " [2] Learn C++ Programming Concepts
";
        cout << " [3] Exit Book System Program
";
        cout << "-----
";
        cout << "Provide Selection Option: ";
        cin >> inputSelection;

        switch (inputSelection) {
            case 1: openHistoryModule(); break;
            case 2: openConceptsModule(); break;
            case 3: cout << "
Closing electronic textbook terminal execution.
"; break;
            default: cout << "
[3]mSelection outside bounds. Please try again. [0m
";

```

```

    }
} while (inputSelection != 3);
return 0;
}

void displayBookHeader() {
    cout << "
[34m-----
";
    cout << "          FOUNDATIONS AND HISTORY OF C++ TEXTBOOK
";
    cout << "----- [0m
";
}

void openHistoryModule() {
    int innerChoice;
    cout << "
=== HISTORY AND GENERATION CHAPTERS ===
";
    cout << " [1] Evolutionary Origins & Standardization Roadmap
";
    cout << " [2] View Structural Language Topology Tree Map
";
    cout << " [3] Return to Main Menu
";
    cout << "Choose option: ";
    cin >> innerChoice;

    if (innerChoice == 1) {
        streamFileContents("history_origin.txt",
            "Default Asset: C++ was created by Bjarne Stroustrup in 1979 at Bell Laboratories
"
            "as an enhancement to C, introducing object-oriented features seamlessly.");
    } else if (innerChoice == 2) {
        drawTimelineTree();
    }
}

```

```

void openConceptsModule() {
    int moduleSelection;
    cout << "
=== CORE PROGRAMMING CONCEPTS ENVIRONMENT ===
";
    cout << " [1] Data Types & Memory Configurations
";
    cout << " [2] Control Flow (if-else, switch-case)
";
    cout << " [3] Iteration Arrays & Matrix Elements
";
    cout << " [4] File System Streams Analysis
";
    cout << "Choose option: ";
    cin >> moduleSelection;

    switch (moduleSelection) {
        case 1:
            streamFileContents("concept_datatypes.txt",
                "Data Types include: Core Primitive elements (int, char, double) and
Modifiers.");
            break;
        case 2:
            cout << "
[Control Flow] Branching systems isolate condition blocks to run specific execution lines.
";
            break;
        case 3:
            cout << "
[Arrays & Loops] Contiguous data structures processed iteratively via for or while layouts.
";
            break;
        case 4:
            cout << "
[File Streams] Input/Output management executed by linking ifstream/ofstream utilities.
";
            break;
        default: cout << "
Invalid selection key variable.
";
    }
}

void streamFileContents(string fileTarget, string fallbackText) {
    ifstream fileStream(fileTarget);
    cout << "
-----
";

    if (fileStream.is_open()) {
        string stringRow;
        while (getline(fileStream, stringRow)) {
            cout << stringRow << endl;
        }
        fileStream.close();
    } else {
        cout << "[SYSTEM NOTICE - EXTERNAL LOG MISSING] Loading local backup metrics:
";

        cout << fallbackText << endl;
    }
    cout << "-----

```

```

";
}

void drawTimelineTree() {
    cout << "
=====
    cout << "          C++ STRUCTURAL GENEALOGY CHART          " << endl;
    cout << "=====
    cout << " Fortran ----> Algol 60 ----> CPL ----> BCPL ----> C " << endl;
    cout << "                                     | " << endl;
    cout << " Simula 67 -----+----> C with Classes" << endl;
    cout << "                                     | " << endl;
    cout << " C with Classes -----> C++ (1983)" << endl;
    cout << "                                     | " << endl;
    cout << " C++ + Ada + Clu + ML -----> C++ ARM" << endl;
    cout << "                                     | " << endl;
    cout << " C++ ARM -----> C++ Standard" << endl;
    cout << "=====
}

```

## BONUS CONTENT: Multi-Dimensional Loop & Matrix Parsing Trace Analysis

The matrix exercise initialization module provided on Page 7 maps an explicit two-dimensional data block array framework into processing layout space:

```
int rollNumbers[4][3] = { 1,1,1, 2,2,2, 3,3,3, 4,4,4 };
```

This statement reserves continuous stack indexing blocks allocating exactly 4 independent rows matching 3 explicit internal value columns. The structural value topology sets up as follows:

- **Row Block index 0:** 1, 1, 1
- **Row Block index 1:** 2, 2, 2
- **Row Block index 2:** 3, 3, 3
- **Row Block index 3:** 4, 4, 4

When executing sequential linear tracking traversal using nested parameters, the inner loop parses all column elements inside a row completely before the outer loop changes rows. This produces a continuous text stream without line breaks:

```
111222333444
```

This execution trace illustrates predictable  $O(\text{Rows} \times \text{Columns})$  iteration tracking and establishes row-major sequential memory storage indexing paths.